



151-0854-00L

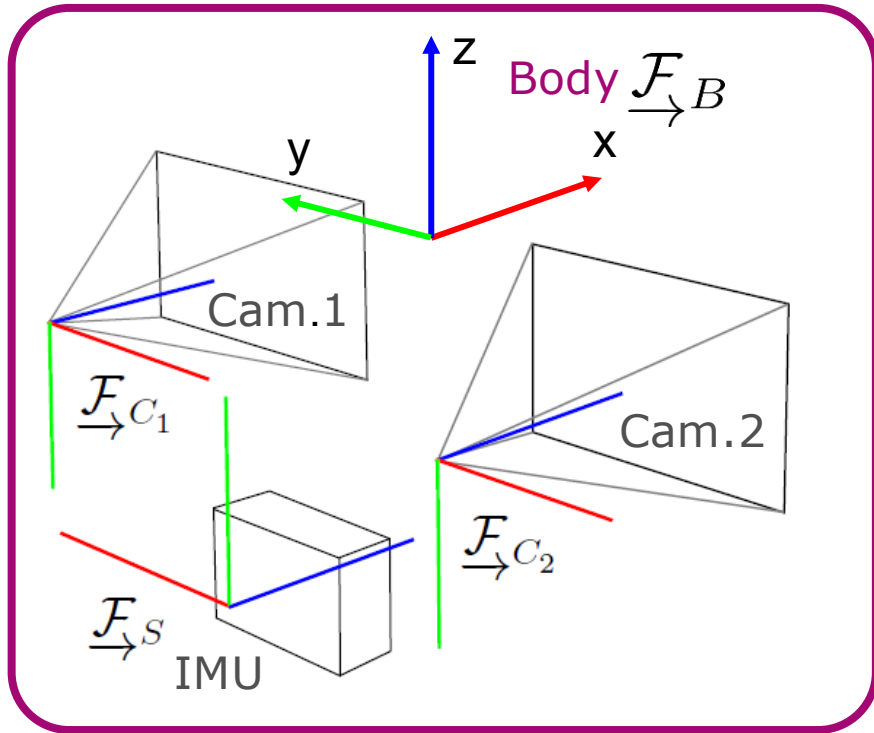
Autonomous Mobile Robots

E02: Kinematics

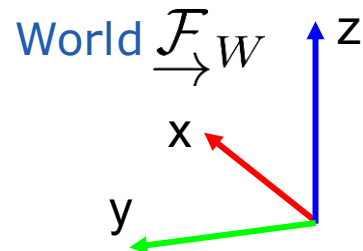
Stefan Leutenegger, Anran Zhang, David Rohr



Frames, Notation, Definitions



A typical example sensor setup



Position vector ${}^W\mathbf{t}_B$:

Expressed in $\mathcal{F}_W$ — End point: origin of $\mathcal{F}_B$ / point B

Alternative writing as **translation vector** ${}^W\mathbf{t}_{WB}$:

Start point: origin of $\mathcal{F}_W$ / point W

Rotation Matrix $\mathbf{R}_{WB}$

Transforms vector coordinate representation as

$${}^W\mathbf{a} = \mathbf{R}_{WB} {}^B\mathbf{a}.$$

Expressed in $\mathcal{F}_W$

Expressed in frame $\mathcal{F}_B$

Velocity vector ${}^W\mathbf{v}_{WB}$

Expressed in $\mathcal{F}_W$

Velocity of frame $\mathcal{F}_B$

Relative to frame $\mathcal{F}_W$

SE3 and Homogenous Transformation Matrices

A 6D pose consists of position and orientation.

We can write the respective change of reference frame of a position vector $\mathbf{t}_P$ as:

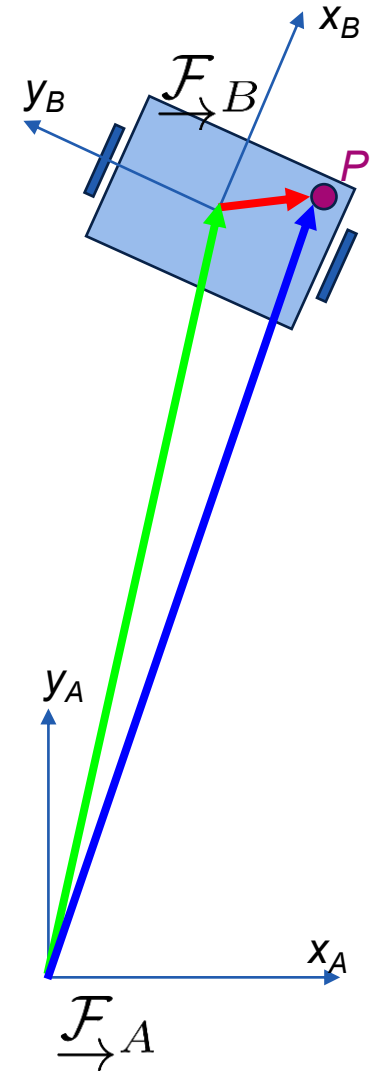
$${}_A\mathbf{t}_P = \mathbf{R}_{AB} {}_B\mathbf{t}_{BP} + {}_A\mathbf{t}_{AB} = {}_A\mathbf{t}_{BP} + {}_A\mathbf{t}_{AB} = {}_A\mathbf{t}_{AP}.$$

Or, we can use the homogeneous transformation matrix: $\mathbf{T}_{AB} = \begin{bmatrix} \mathbf{R}_{AB} & {}_A\mathbf{t}_B \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}.$

$$\text{Now: } {}_A\mathbf{t}_P := \begin{bmatrix} {}_A\mathbf{t}_P \\ 1 \end{bmatrix} = \mathbf{T}_{AB} \begin{bmatrix} {}_B\mathbf{t}_P \\ 1 \end{bmatrix}.$$

$$1. \text{ As inverse transform, we obtain } \mathbf{T}_{BA} = \mathbf{T}_{AB}^{-1} = \begin{bmatrix} \mathbf{R}_{AB}^T & -\mathbf{R}_{AB}^T {}_A\mathbf{t}_B \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}.$$

$$2. \text{ We can stack transformations as } \mathbf{T}_{AC} = \mathbf{T}_{AB} \mathbf{T}_{BC}.$$



Kinds of Manipulators



6 DoF industrial Manipulator



SCARA Robot with 4 DoF, for electronic assembly



Linear Axis Robot



Delta-Robot with 6 parallel Links, 3 DoF

[1] Peter Corke. "Robotics, Vision and Control | Fundamental Algorithms in Python"

Kinematics vs. Dynamics

Kinematics

- Analyses the **motion** of objects **not** considering **causes**.
- Use of rigid body (coordinate frame) **transformations** (pose parameters) and their **derivatives**.

Dynamics

- Considers effects of **forces** and **torques** on **motion**.
- Use of **Newton's Second Law** (or Lagrangian Dynamics, not covered in this course).

Control

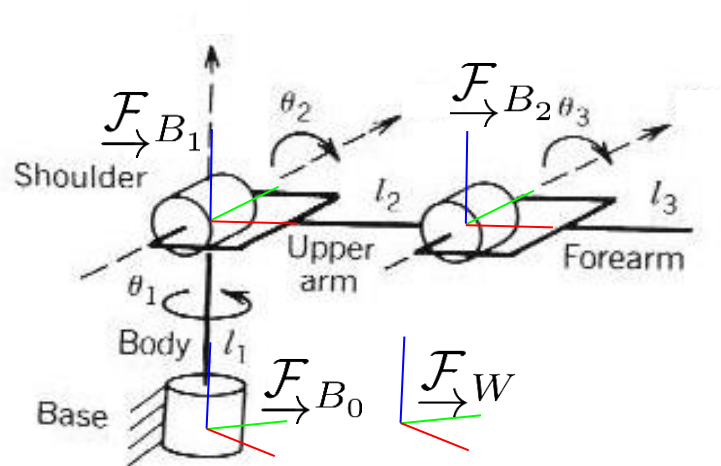
- Physically meaningful models: torques/forces inputs (use dynamics).
- But sometimes, taking pose parameters and/or their derivatives as inputs may be a good enough approximation (use kinematics).

Estimation

- We may have access to / consider either pose parameters / their derivatives or torques (use kinematics or dynamics).

Manipulator Kinematics

Joint angles/positions uniquely define the configuration:



www.tareksobh.org



www.directindustry.com

www.photonics.com

Forward Kinematics $T_{WB_n}(\theta) = ?$

Attach a frame to each rigid body (origin through joint axis) and parameterise the relative transformations $T_{B_{i-1}B_i}(\theta_i)$:

$$T_{WB_n}(\theta) = T_{WB_0} T_{B_0B_1}(\theta_1) \dots T_{B_{n-1}B_n}(\theta_n).$$

Inverse Kinematics $\theta(T_{WB_n}) = ?$

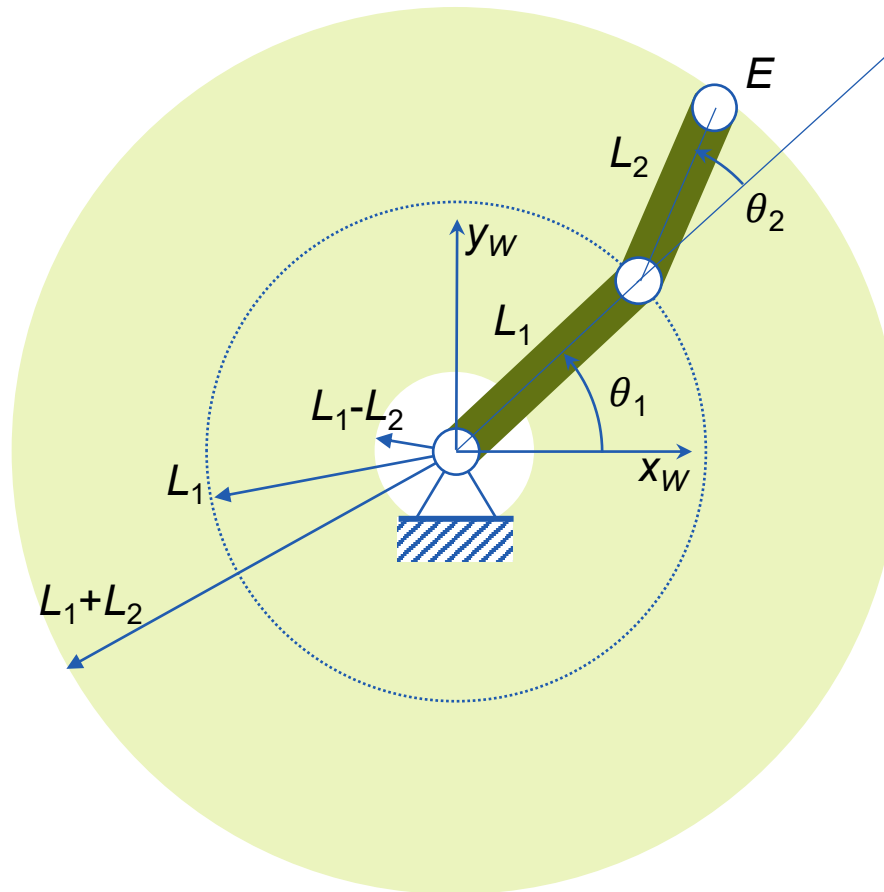
- Often nonlinear, without analytic solution.
- Depending on the configuration (importantly no. joints): under- or over-determined.
- Use numeric (iterative) solving...

Closed Kinematic Chains / Parallel Robots

Parallel kinematics chains, joined by passive links (yielding additional constraints)

Forward Kinematics: 2R Planar Robot Example

What is the end effector pose for a given configuration? $T_{WE} = f(\theta)$



End effector position / orientation in base-coordinates:

$${}_W\mathbf{t}_{WE} = \begin{bmatrix} L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix}.$$
$$\theta_{WE} = \theta_1 + \theta_2$$

Workspace (position) $\mathcal{W}$ for

$$\theta_1 \in [-\pi, \pi], \theta_2 \in [-\pi, \pi].$$

See task 1.1 in exercise (for 3R robot)

Planar 2R Robot | Inverse Kinematics (IK)

Inverse Kinematics regarding position $\theta({}_W\mathbf{t}_{WE}) = ?$

- ${}_W\mathbf{t}_{WE} \notin \mathcal{W}$: no solution!
- ${}_W\mathbf{t}_{WE} \in \mathcal{W}$: at least one solution.

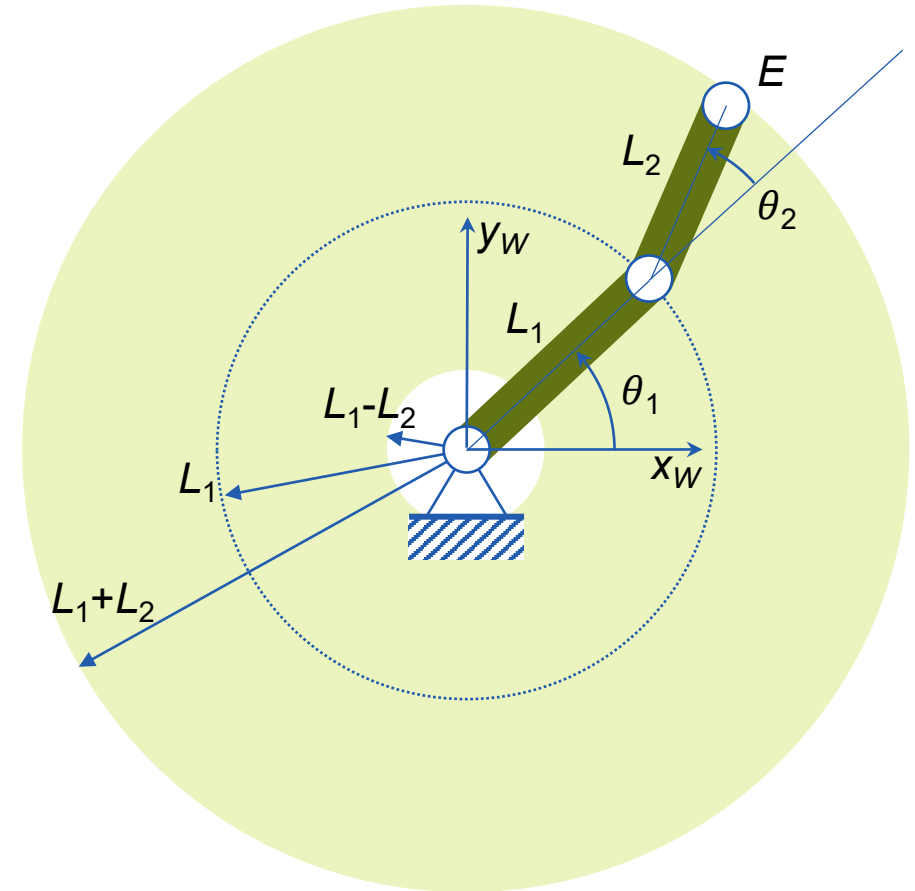
Naive approach:

Solve Forward Kinematics for θ_1, θ_2 :

$$\mathbf{t}_{WE} = \begin{bmatrix} L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix}.$$

$\Rightarrow$ Analytically not trivial / feasible in general

$\Rightarrow$ Instead: try iteratively



Planar 2R Robot | Inverse Kinematics (IK)

Inverse Kinematics regarding position $\theta({}_W\mathbf{t}_{WE}) = ?$

Iterative IK:

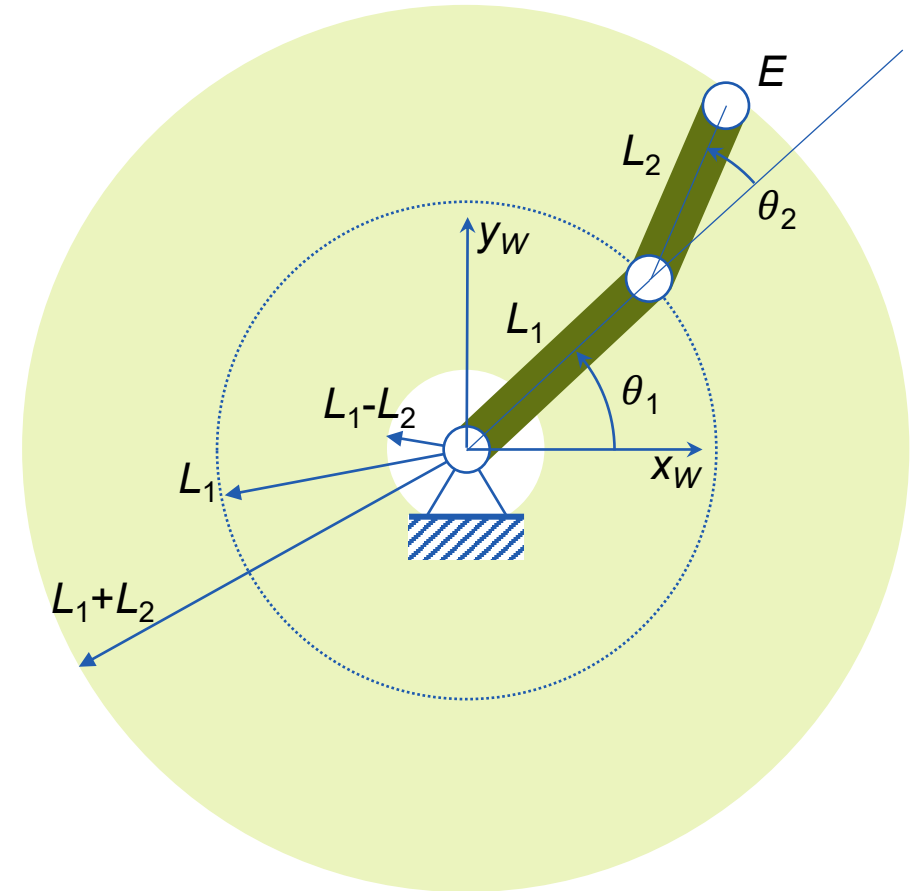
- End-effector position Jacobian $\mathbf{J}(\theta)$

$${}_W\mathbf{v}_{WE} = \partial {}_W\mathbf{t}_{WE} / \partial \theta \dot{\theta} := \mathbf{J}(\theta) \dot{\theta} \quad \text{See tasks 1.2-2.2}$$

- Iterate until convergence $\|{}_W\mathbf{t}_{WE, goal} - {}_W\mathbf{t}_{WE}\|_2 < \epsilon$

$$\theta \leftarrow \theta + \underbrace{\mathbf{J}(\theta)^\dagger k_p ({}_W\mathbf{t}_{WE, goal} - {}_W\mathbf{t}_{WE})}_{\text{incremental step towards goal}} \quad \text{See task 3}$$

incremental step in joint angles



Planar 2R Robot | Inverse Kinematics (IK)

Inverse Kinematics regarding position $\theta({}_W\mathbf{t}_{WE}) = ?$

Iterative IK:

- End-effector position Jacobian $\mathbf{J}(\theta)$

$${}_W\mathbf{v}_{WE} = \partial {}_W\mathbf{t}_{WE} / \partial \theta \dot{\theta} = \mathbf{J}(\theta) \dot{\theta} \quad \text{See tasks 1.2-2.2}$$

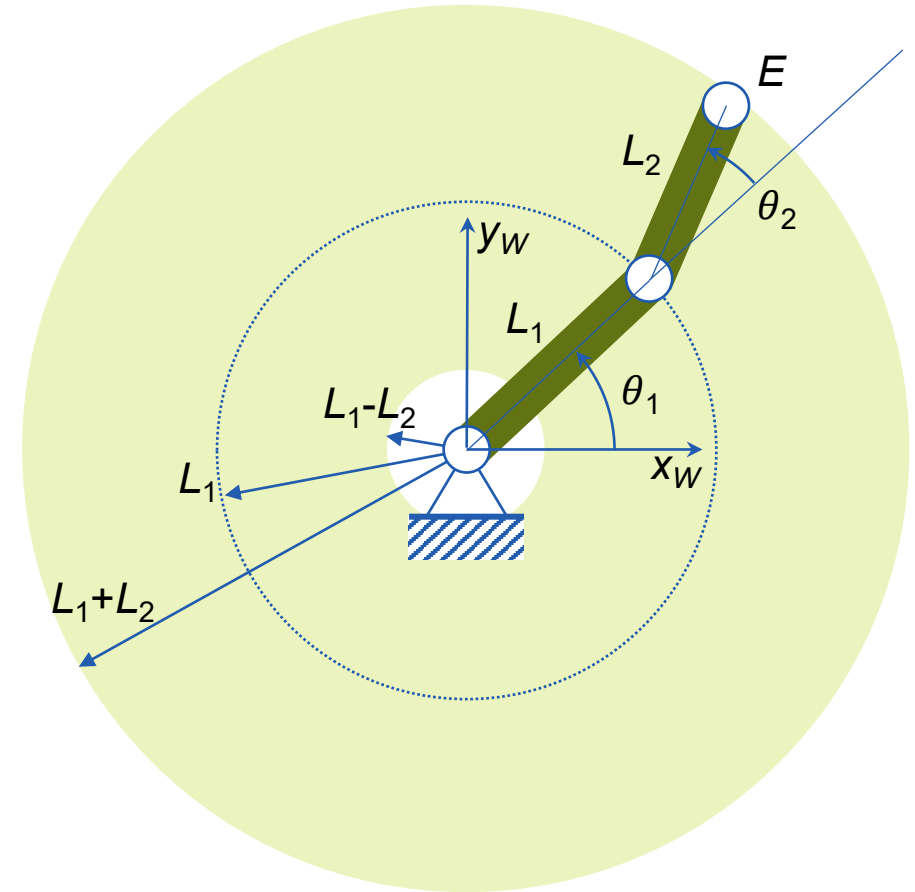
- Iterate until convergence $\|{}_W\mathbf{t}_{WE, goal} - {}_W\mathbf{t}_{WE}\|_2 < \epsilon$

$$\theta \leftarrow \theta + \mathbf{J}(\theta)^\dagger k_p ({}_W\mathbf{t}_{WE, goal} - {}_W\mathbf{t}_{WE}) \quad \text{See task 3}$$

using pseudoinverse: $\mathbf{J}(\theta)^\dagger = \mathbf{J}(\theta)^T (\mathbf{J}(\theta)\mathbf{J}(\theta)^T)^{-1}$

On pseudoinverse for solving $Ax = b$:

- if no sol. exists: $A^\dagger b = \underset{x}{\operatorname{argmin}} \|Ax - b\|_2$
- if mult. sols. exist: $A^\dagger b = \underset{x}{\operatorname{argmin}} \|x\|_2 \text{ s.t. } Ax = b$



Over- & Underactuated (redundant) Manipulators

Underactuated

- Workspace often not full 6D
⇒ We can solve IK nevertheless.



[1], Adept Cobra s600, Adept Technology, Inc.)

Overactuated

- In general, we have infinitely many IK solutions.
- Reduces singularities
- Better ability to avoid self-collision
- We can minimize energy/time when moving from one configuration to the next.
- Workspace often not analytically computable.
- IK solutions often only numerically computable.

Over- & Underactuated (redundant) Manipulators

Underactuated

- Workspace often not full 6D
⇒ We can solve IK nevertheless.



[1], Adept Cobra s600, Adept Technology, Inc.)

Overactuated



Thanks! Questions?

E02: Python environment setup

Via Conda:

- <https://www.anaconda.com/docs/getting-started/miniconda/install#quickstart-install-instructions>
- `source ~/miniconda3/bin/activate && conda init --all`, restart terminal

Task	Windows (Anaconda/Miniforge Prompt)	macOS / Linux (Terminal)
Create Env	<code>conda create -n myenv python=3.10</code>	<code>conda create -n myenv python=3.10</code>
Activate	<code>conda activate myenv</code>	<code>conda activate myenv</code> (or <code>source activate myenv</code>)
Deactivate	<code>conda deactivate</code>	<code>conda deactivate</code>
List Envs	<code>conda env list</code>	<code>conda env list</code>
Install	<code>conda install yourpackages -c conda-forge</code>	<code>pip install -r requirements.txt</code>

E02: Python environment setup

Via virtual environment (venv):

Feature	Windows	macOS	Linux (Ubuntu/Debian)
Command Prefix	<code>python</code> or <code>py</code>	<code>python3</code>	<code>python3</code>
Create Cmd	<code>python -m venv .venv</code>	<code>python3 -m venv .venv</code>	<code>python3 -m venv .venv</code>
Activate Cmd	<code>.venv\Scripts\activate</code>	<code>source .venv/bin/activate</code>	<code>source .venv/bin/activate</code>
Pre-installed?	Yes (if installed via python.org)	Yes (via Xcode Tools)	No (requires <code>apt install</code>)
Extra Step	May need Execution Policy fix	None	<code>sudo apt install python3-venv</code>
Verify Path	<code>where python</code>	<code>which python</code>	<code>which python</code>
Deactivate	<code>deactivate</code>	<code>deactivate</code>	<code>deactivate</code>
Install	<code>pip install -r requirements.txt</code>	<code>pip install -r requirements.txt</code>	<code>pip install -r requirements.txt</code>

E02: Python environment setup

Jupyter Notebooks

Run directly in VS Code (Jupyter extension)

OR

`pip/conda install notebook && jupyter notebook` (launches a window in browser, then select file and go through the blocks. Can run blocks with shift+enter)